

Memorando

De: Emanuel Carneiro dos Santos

Nº de Matrícula: 20230429

Disciplina: Computação Paralela e Distribuída

Assunto: Introdução ao Intel VTune

Data: 15/04/2026

1. Introdução

O presente trabalho tem como objectivo analisar o desempenho de programas OpenMP e otimizar a sua execução paralela, recorrendo à ferramenta de **profiling** Intel® VTune™ Profiler. Pretende-se, assim, identificar os pontos críticos (**hotspots**) e os gargalos (**bottlenecks**) em três implementações distintas de uma rotina de multiplicação de matrizes, compreendendo de que forma as directivas OpenMP influenciam o desempenho e a eficiência do programa.

O **Intel® VTune™ Profiler** é uma ferramenta de análise de desempenho desenvolvida pela Intel, direccionada a plataformas baseadas em arquitectura x86. Suporta diversas linguagens de programação, como C/C++, Python e Java, bem como modelos de programação paralela como OpenMP e MPI. A sua principal utilidade reside na capacidade de identificar funções que consomem mais tempo de execução, medir sobrecargas em programas multithread, e recolher métricas de hardware detalhadas, tais como consumo de energia, acessos à memória e instruções por ciclo de relógio. A ferramenta pode ser utilizada tanto através de uma interface gráfica (**GUI**) como por linha de comandos (**CLI**), tornando-a flexível para diferentes ambientes de trabalho.

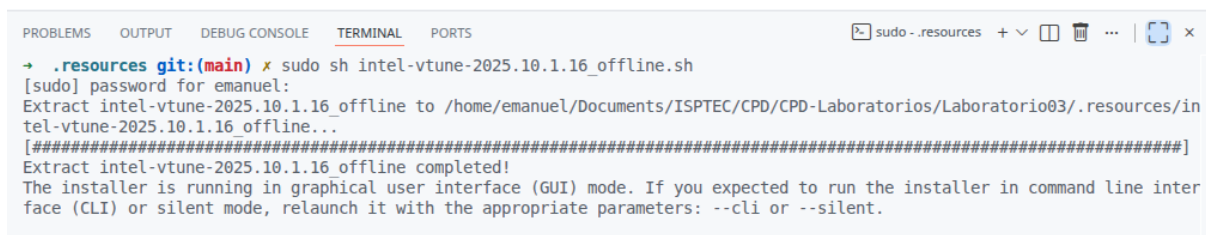
O **OpenMP** (Open Multi-Processing) é uma API para programação paralela em memória partilhada, amplamente utilizada em C, C++ e Fortran. Através de directivas de compilador (pragmas), permite ao programador paralelizar ciclos e regiões de código de forma relativamente simples. Contudo, uma paralelização incorrecta pode introduzir sobrecarga significativa, nomeadamente através de sincronização excessiva, desequilíbrio de carga entre threads ou criação repetida de regiões paralelas, degradando o desempenho em vez de o melhorar.

No acto da experiência laboratorial, foram analisadas três implementações de multiplicação de matrizes em OpenMP — `mult-sync.c`, `mult-redux.c` e `mult-good.c` — utilizando as análises de Hotspots e Threading do Intel VTune Profiler. Procedeu-se à compilação de cada programa com a flag de depuração (`-g`), executou-se a análise de Hotspots para identificar as secções de código mais exigentes, e a análise de Threading para avaliar a eficiência da utilização dos núcleos do processador. Foram identificados problemas de sincronização excessiva e de criação repetida de regiões paralelas nas versões menos eficientes, contrastando com a abordagem correcta da versão `mult-good.c`.

2. Experiências Realizadas

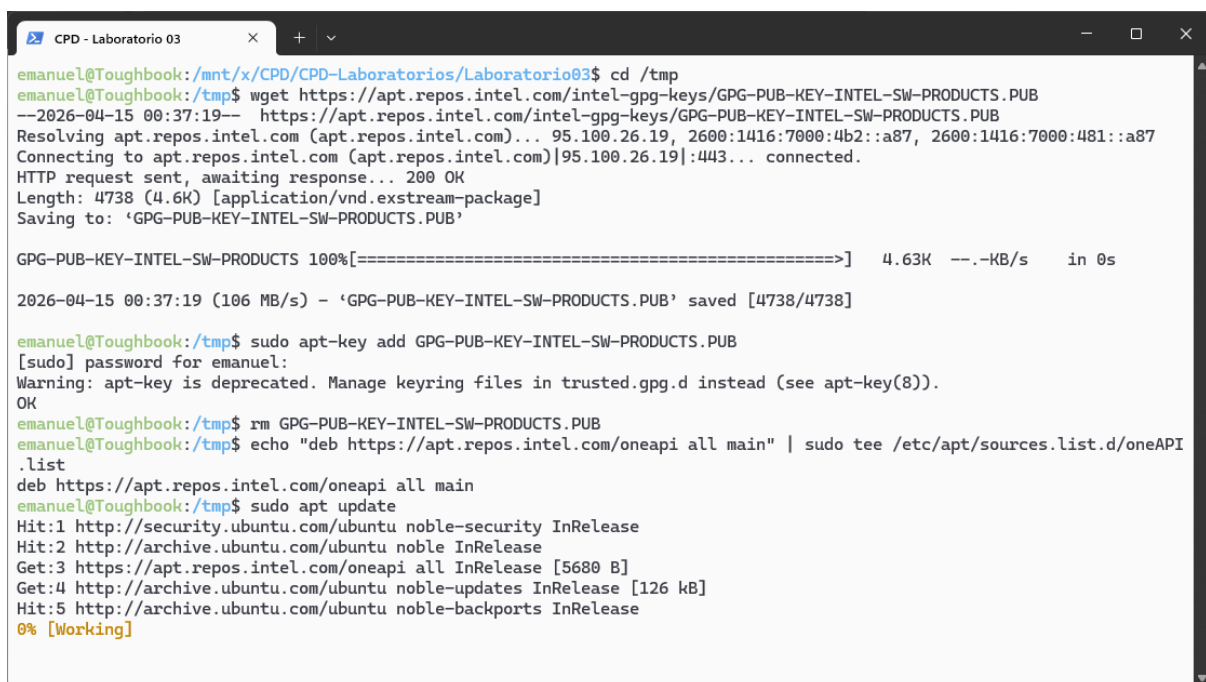
2.1. Configuração do Ambiente e Compilação

A experiência foi realizada em ambiente Linux. Primeiramente foi baixado e instalado a ferramenta principal de trabalho Intel Vtune.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
→ .resources git:(main) ✖ sudo sh intel-vtune-2025.10.1.16_offline.sh
[sudo] password for emmanuel:
Extract intel-vtune-2025.10.1.16 offline to /home/emmanuel/Documents/ISPTEC/CPD/CPD-Laboratorios/Laboratorio03/.resources/in
tel-vtune-2025.10.1.16 offline...
[#####]
Extract intel-vtune-2025.10.1.16 offline completed!
The installer is running in graphical user interface (GUI) mode. If you expected to run the installer in command line inter
face (CLI) or silent mode, relaunch it with the appropriate parameters: --cli or --silent.
```

Figura 1 – Executando o script de download do Intel VTune.



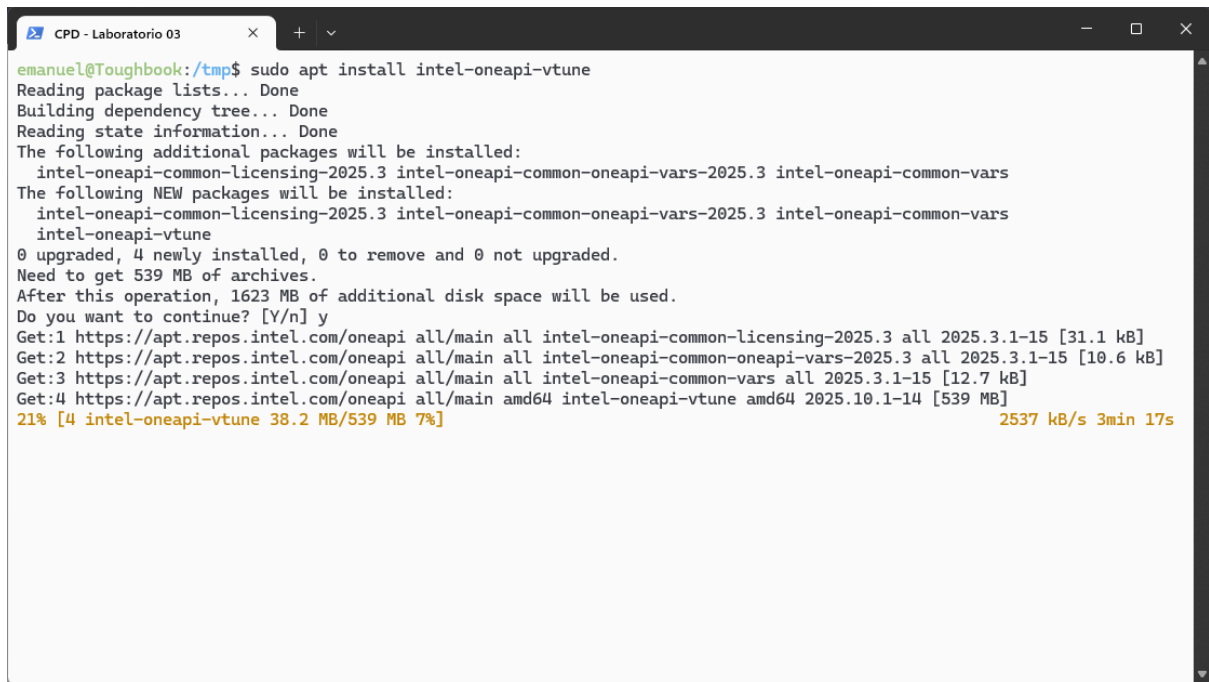
```
CPD - Laboratorio 03
emmanuel@Toughbook: /mnt/x/CPD/CPD-Laboratorios/Laboratorio03$ cd /tmp
emmanuel@Toughbook: /tmp$ wget https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB
--2026-04-15 00:37:19-- https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB
Resolving apt.repos.intel.com (apt.repos.intel.com)... 95.100.26.19, 2600:1416:7000:4b2::a87, 2600:1416:7000:481::a87
Connecting to apt.repos.intel.com (apt.repos.intel.com)[95.100.26.19]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4738 (4.6K) [application/vnd.exstream-package]
Saving to: 'GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB'

GPG-PUB-KEY-INTEL-SW-PRODUCTS 100%[=====] 4.63K --.-KB/s in 0s

2026-04-15 00:37:19 (106 MB/s) - 'GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB' saved [4738/4738]

emmanuel@Toughbook: /tmp$ sudo apt-key add GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB
[sudo] password for emmanuel:
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
emmanuel@Toughbook: /tmp$ rm GPG-PUB-KEY-INTEL-SW-PRODUCTS.PUB
emmanuel@Toughbook: /tmp$ echo "deb https://apt.repos.intel.com/oneapi all main" | sudo tee /etc/apt/sources.list.d/oneAPI
.list
deb https://apt.repos.intel.com/oneapi all main
emmanuel@Toughbook: /tmp$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 https://apt.repos.intel.com/oneapi all InRelease [5680 B]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
0% [Working]
```

Figura 2 – Baixando e Configurando as assinaturas do VTune.



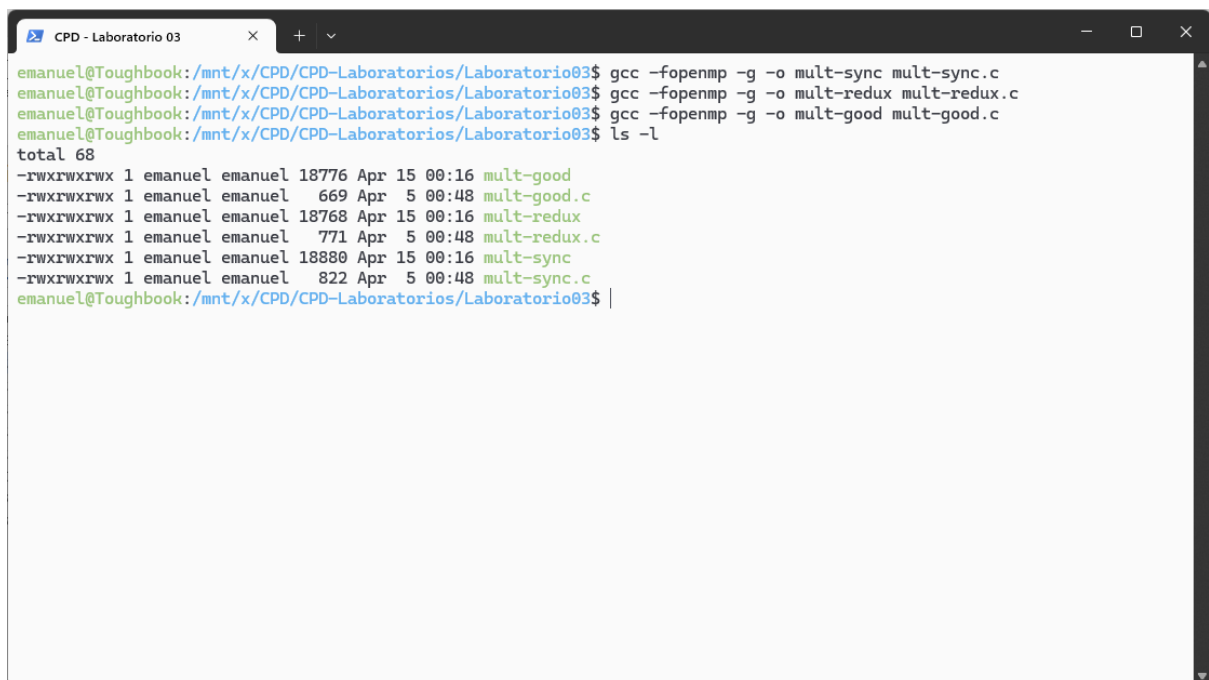
```
CPD - Laboratorio 03
emanuel@Toughbook:/tmp$ sudo apt install intel-oneapi-vtune
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  intel-oneapi-common-licensing-2025.3 intel-oneapi-common-oneapi-vars-2025.3 intel-oneapi-common-vars
The following NEW packages will be installed:
  intel-oneapi-common-licensing-2025.3 intel-oneapi-common-oneapi-vars-2025.3 intel-oneapi-common-vars
  intel-oneapi-vtune
0 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 539 MB of archives.
After this operation, 1623 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 https://apt.repos.intel.com/oneapi all/main all intel-oneapi-common-licensing-2025.3 all 2025.3.1-15 [31.1 kB]
Get:2 https://apt.repos.intel.com/oneapi all/main all intel-oneapi-common-oneapi-vars-2025.3 all 2025.3.1-15 [10.6 kB]
Get:3 https://apt.repos.intel.com/oneapi all/main all intel-oneapi-common-vars all 2025.3.1-15 [12.7 kB]
Get:4 https://apt.repos.intel.com/oneapi all/main amd64 intel-oneapi-vtune amd64 2025.10.1-14 [539 MB]
21% [4 intel-oneapi-vtune 38.2 MB/539 MB 7%] 2537 kB/s 3min 17s
```

Figura 3 – Instalação do Intel Vtune no Linux.

Para associar correctamente as medições ao código-fonte, os programas foram compilados com a flag de depuração `-g`, tal como indicado na documentação da ferramenta. Os comandos utilizados foram os seguintes:

```
gcc -fopenmp -g -o mult-sync mult-sync.c
gcc -fopenmp -g -o mult-reduce mult-reduce.c
gcc -fopenmp -g -o mult-good mult-good.c
```

A Figura 3 apresenta a captura de ecrã da compilação dos três programas no terminal.



```
CPD - Laboratorio 03
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio03$ gcc -fopenmp -g -o mult-sync mult-sync.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio03$ gcc -fopenmp -g -o mult-reduce mult-reduce.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio03$ gcc -fopenmp -g -o mult-good mult-good.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio03$ ls -l
total 68
-rwxrwxrwx 1 emanuel emanuel 18776 Apr 15 00:16 mult-good
-rwxrwxrwx 1 emanuel emanuel 669 Apr 5 00:48 mult-good.c
-rwxrwxrwx 1 emanuel emanuel 18768 Apr 15 00:16 mult-reduce
-rwxrwxrwx 1 emanuel emanuel 771 Apr 5 00:48 mult-reduce.c
-rwxrwxrwx 1 emanuel emanuel 18880 Apr 15 00:16 mult-sync
-rwxrwxrwx 1 emanuel emanuel 822 Apr 5 00:48 mult-sync.c
emanuel@Toughbook:/mnt/x/CPD/CPD-Laboratorios/Laboratorio03$
```

Figura 4 – Compilação dos programas `mult-sync.c`, `mult-reduce.c` e `mult-good.c` com suporte a OpenMP e símbolos de depuração.

2.2. Análise de Hotspots

A análise de Hotspots foi executada com Sampling em Modo de Utilizador sobre a implementação serial do projecto. Esta análise permitiu identificar as secções de código onde o programa despende a maior parte do tempo de execução. Na tabela de Principais Hotspots, verificou-se que a função main concentrava praticamente todo o tempo de CPU, especificamente nos ciclos de multiplicação das matrizes. A visualização Flame Graph confirmou este resultado, com o bloco correspondente ao ciclo mais interno a ocupar a maior área do gráfico.

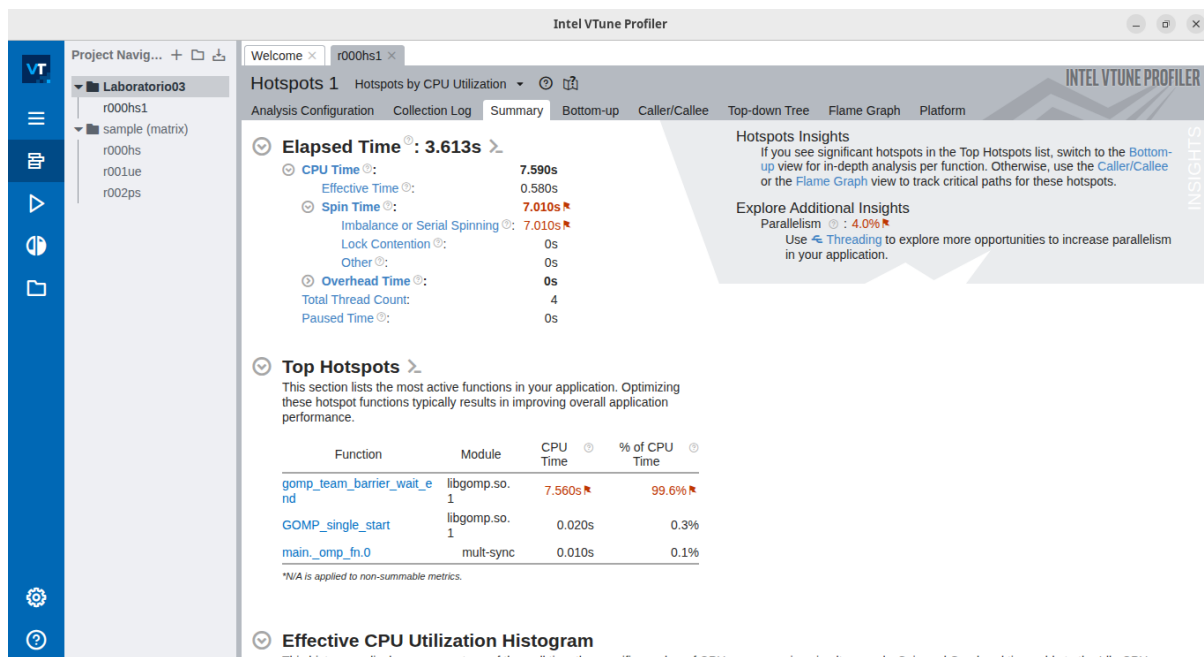


Figura 5 – Análise de Hotspots: tabela de Principais Hotspots com o tempo por função.

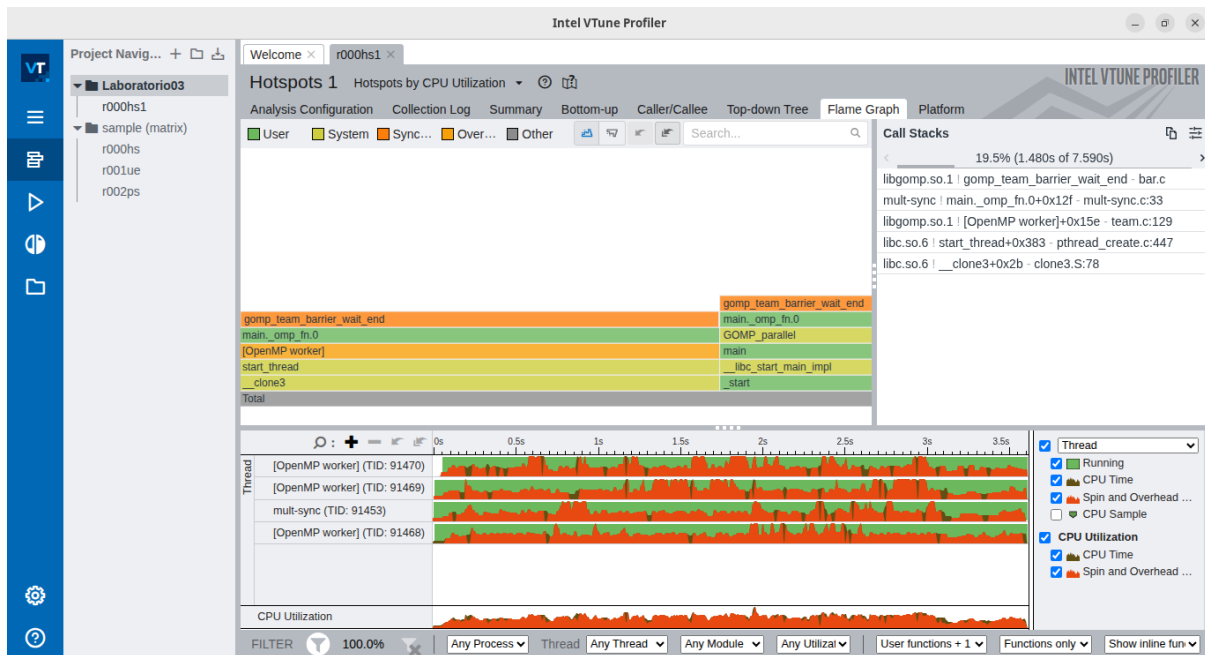


Figura 6 – Análise de Hotspots: visualização Flame Graph com proporção do tempo por rotina.

2.3. Análise de Threading – mult-sync.c

Ao abrir o ficheiro `mult-sync.c`, identificaram-se dois problemas principais. Em primeiro lugar, a região paralela (`#pragma omp parallel`) é declarada fora dos ciclos externos, mas as directivas `#pragma omp single` e `#pragma omp for reduction` são utilizadas dentro dos ciclos `i` e `j`, o que implica que para cada par (i, j) — ou seja, $1.000 \times 1.000 = 1.000.000$ de iterações — ocorre uma sincronização explícita através do `#pragma omp barrier`. Em segundo lugar, o uso de `#pragma omp single` para inicializar `acc` e escrever o resultado em `C[j + i*N]` força todos os outros threads a ficarem bloqueados em espera, gerando um overhead de sincronização muito elevado.

Na análise de Threading com o VTune, a utilização efectiva da CPU foi muito baixa. A visualização Top-down Tree, expandindo `Total > clone > start thread > [OpenMP worker] > main.omp_fn.0`, revelou que as directivas `#pragma omp single` e `#pragma omp barrier` apresentavam overhead dominante. O principal problema é, portanto, a sincronização excessiva e o bloqueio de threads dentro de um ciclo extremamente frequente.

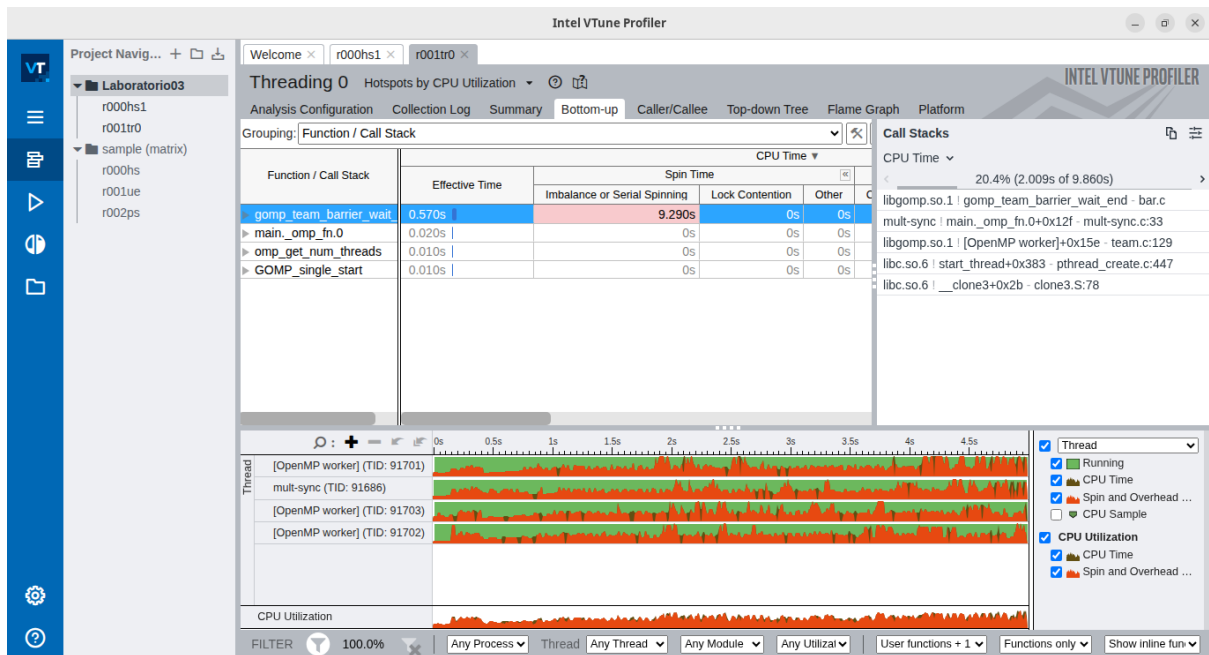


Figura 7 – Análise de Threading do mult-sync: visualização Top-down Tree evidenciando o overhead das directivas de sincronização.

2.4. Análise de Threading – mult-reduce.c

Em mult-reduce.c, o problema identificado é distinto. A região paralela é criada dentro dos dois ciclos externos (i e j), o que significa que para cada célula $C[j + i*N]$ é criada e destruída uma nova equipa de threads. Esta operação é extremamente dispendiosa: a criação de threads envolve alocação de recursos pelo sistema operativo, sincronização inicial, e distribuição de tarefas — repetida 1.000.000 de vezes. O ponto de sincronização no código é a própria barreira implícita no final de `#pragma omp for reduction`, complementada pelo encerramento da região paralela.

O programa gastou muito tempo nesse ponto porque a construção e destruição repetida de regiões paralelas implica custos de sincronização muito superiores ao trabalho útil efectuado em cada invocação, que corresponde apenas a $N = 1000$ operações de multiplicação e adição.

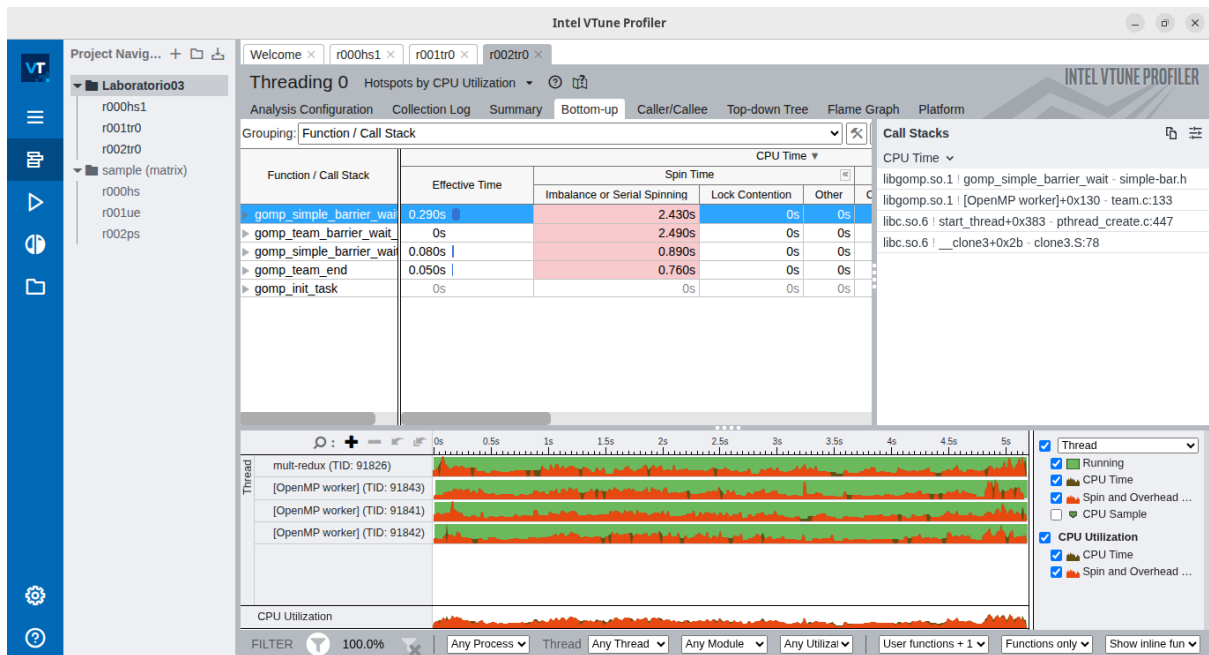


Figura 8 – Análise de Threading do mult-redux: overhead causado pela criação repetida de regiões paralelas dentro dos ciclos externos.

2.5. Análise de Threading – mult-good.c

Em mult-good.c, a directiva `#pragma omp parallel for private(j, k)` é aplicada directamente ao ciclo mais externo (i), criando a região paralela uma única vez e distribuindo as iterações de i pelos threads disponíveis. Desta forma, cada thread executa um bloco de linhas da matriz C de forma completamente independente, sem necessitar de sincronização durante o cálculo. A ausência de barreiras internas, de directivas `single`, e a criação de apenas uma região paralela resulta numa utilização efectiva da CPU muito superior, com tempo de spin e overhead mínimos, confirmando esta como a implementação mais eficiente das três.

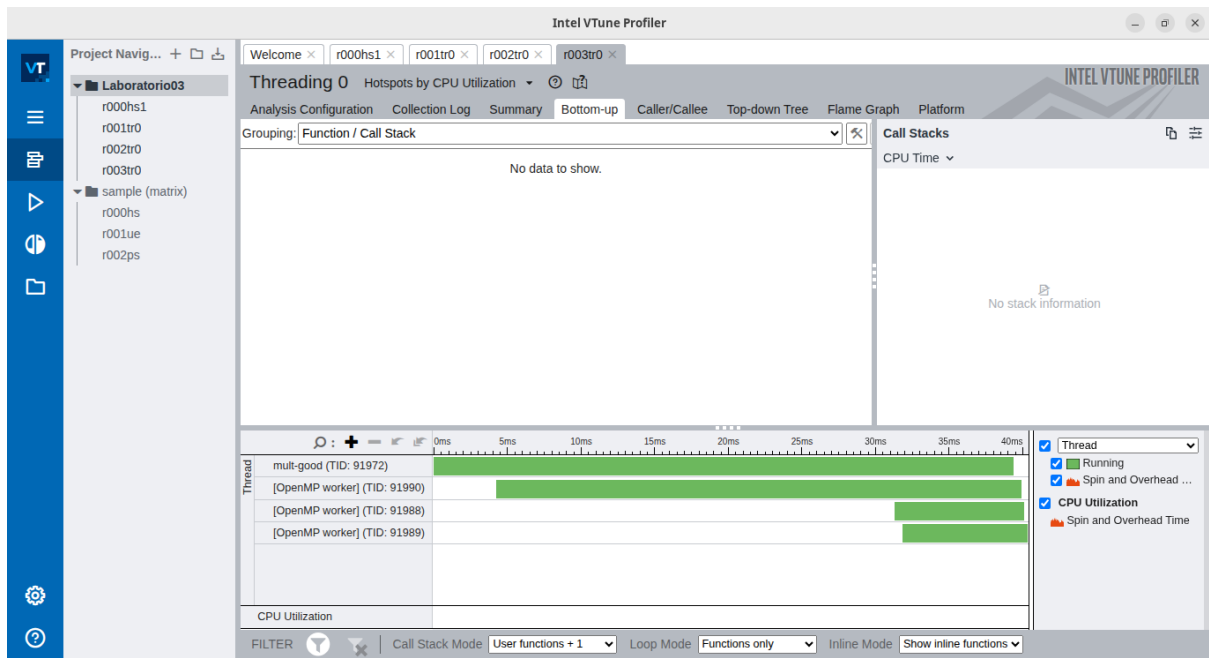


Figura 9 – Análise de Threading do mult-good: utilização efectiva da CPU e tempo de spin reduzido face às versões anteriores.

A Tabela 1 resume os problemas identificados em cada implementação e a respectiva causa.

Programa	Problema Identificado	Causa
mult-sync.c	Sincronização excessiva com omp single e omp barrier dentro do ciclo (i, j)	Overhead de barreira repetido 10^6 vezes; threads em espera ociosa
mult-redux.c	Criação de região paralela 10^6 vezes (dentro dos ciclos i e j)	Custo de criação/destruição de threads supera o trabalho útil por iteração
mult-good.c	Sem problemas relevantes – implementação eficiente	Região paralela criada apenas uma vez; distribuição das iterações de i pelos threads

Tabela 1 – Comparação dos problemas de desempenho identificados nas três implementações OpenMP.

3. Desafios

O principal desafio desta aula laboratorial consistiu em compreender, a partir dos resultados do VTune, de que forma pequenas diferenças nas directivas OpenMP produzem impactos tão distintos no desempenho. A leitura das visualizações Bottom Up, Top-down Tree e Flame Graph exige familiarização com a ferramenta, nomeadamente a interpretação das métricas de Effective Time, Spin Time e Overhead Time.

A instalação dos símbolos de depuração da libgomp no Ubuntu (conforme descrito no Apêndice A do guia laboratorial) foi igualmente um passo adicional necessário para que o VTune apresentasse correctamente as funções OpenMP na árvore de chamadas, sem o que várias funções apareceriam como [Unknown].

4. Referências Bibliográficas

Intel Corporation. *Intel® VTune™ Profiler User Guide*. Disponível em: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2025-1/overview.html>. Acedido em: 14 de Abril de 2026.

Intel Corporation. *Intel® VTune™ Profiler Installation Guide 2025.0*. Disponível em: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/installation-guide/2025-0/overview.html>. Acedido em: 14 de Abril de 2026.

Intel Corporation. *Get Intel® VTune™ Profiler*. Disponível em: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler-download.html>. Acedido em: 14 de Abril de 2026.

OpenMP Architecture Review Board. *OpenMP Application Programming Interface – Version 4.5*. Disponível em: <http://www.openmp.org/wp-content/uploads/openmp-4.5.pdf>. Acedido em: 14 de Abril de 2026.

Intel Corporation. *Intel® VTune™ Profiler Performance Analysis Cookbook 2025.0*. Disponível em: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/cookbook/2025-0/overview.html>. Acedido em: 14 de Abril de 2026.

Intel Corporation. *Intel® VTune™ Profiler System Requirements*. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/system-requirements/vtune-profiler-system-requirements.html>. Acedido em: 14 de Abril de 2026.

Costa, J. *Computação Paralela e Distribuída – Aula de Laboratório Nº 3*. ISPTEC, 2021–22.

5. Repositório GitHub

Repositório: github.com/SW-Wanted/CPD-Laboratorios.¹

¹ Foi enviado um convite de colaborador ao utilizador [joaojdacosta](#)